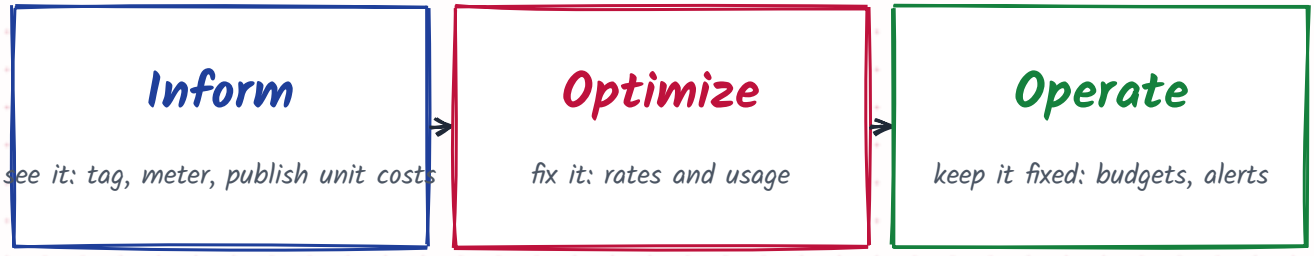
Cost and FinOps

Lesson 11 of 12 · Where the money really goes, and which lever to pull first.

In this lesson → ① The Fin Ops loop ② The real price of a GPU ③ Ways to buy ④ The token bill and five levers ⑤ Self-host or API ⑥ Attribution and alerts ⑦ Hidden lines
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 The Fin Ops loop



1-0-0: see it, fix it, keep it fixed

> **1-0-0**: Inform (see it: tag GPUs, meter tokens), Optimise (fix rate and usage), Operate (keep it fixed: budgets, alerts). Cost becomes a design input, like latency.

→ 98% of teams manage some AI spend (63% a year earlier, State of FinOps 2026)
→ **Cost per token** = efficiency. **Cost per resolved case** = outcome (ranked higher)
→ **FOCUS**: one open billing format; native token tracking is only a roadmap item, so use a gateway or provider usage APIs

CLOUD NOTEBOOK

2 Measure utilisation honestly

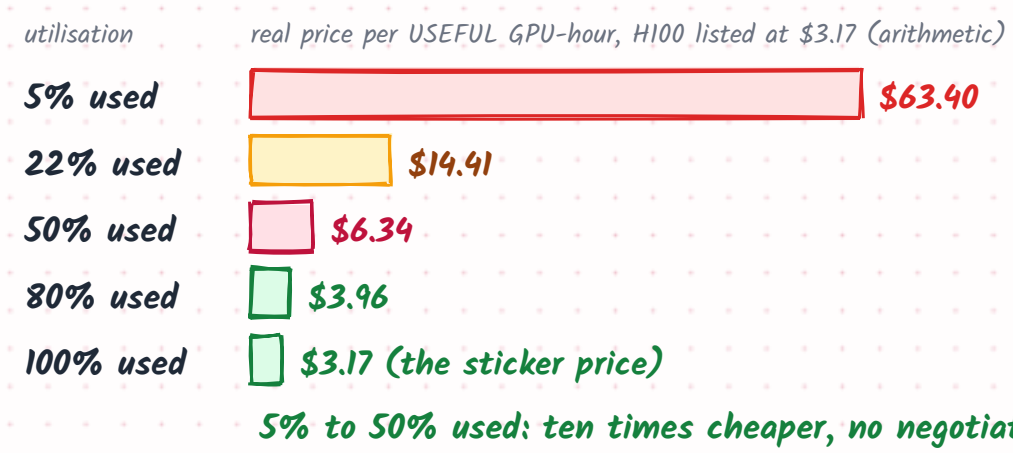
Reading	Means
nvidia-smi GPU-Util 100%	a kernel ran at some point in the sample window
DCGM SM active 0.13	2 of 16 processing blocks in use
DCGM SM active 1.0	all 16 in use

```
# the misleading number
nvidia-smi --query-gpu=index,utilization.gpu,memory.used --format=csv -l 1
# better: DCGM profiling fields (SM active, occupancy, tensor pipe)
dcgmi dmon -e 203,1002,1003,1004
# allocated but idle for 30 min (Prometheus + dcgm-exporter)
avg_over_time(DCGM_FI_PROF_SM_ACTIVE[30m]) < 0.1
```

SM active at or above 0.8 is **necessary, not sufficient**. How idle is normal? Vendor telemetry (2026): about **5%** average across unoptimised Kubernetes clusters. Survey: 86% of enterprises with their own GPUs report 50% or less (self-reported).

3 The real price

Real price = list $/GPU-hr ÷ utilisation. Arithmetic, H100 at $3.17/hr (index, Oct 2026). Script: l11_gpu_price.py.



5% to 50% used: ten times cheaper, no negotiating

Use	$3.17 H100	$6.88 H100
5%	$63.40	$137.60
22%	$14.41	$31.27
50%	$6.34	$13.76
80%	$3.96	$8.60
100%	$3.17	$6.88

5% to 50% = **10x** cheaper. Best rate discounts: about 2 to 3x. **Fix idle before you negotiate price.**

4 Right-size, then share

Weights + KV cache fit in 10-20 GB (for example a 7B model at FP8 or INT4)? A full 80 GB card is oversized: use a MIG slice or an L4 / L40S-class card.

Method	What you get	Use for
MIG	own memory, fault isolation, up to 7 slices	production multi-tenant serving
time-slicing	pods take turns, no isolation	trusted dev and test
MPS	kernels from several processes at once	trusted workloads

- ✓ Bin-pack replicas by GPU **memory**, not one pod per GPU
- ✓ Idle alert: SM active under 0.1 for 30 min on an allocated GPU
- ✓ Scale to zero for bursty endpoints (cold start is the price)
- ✓ Charge idle GPU cost to the team holding the GPU

5 Ways to buy GPU time

Option	Good for	Catch
on-demand	bursty experiments	highest rate, capacity not guaranteed
neocloud	training, cheap inference	egress, minimum commits, thinner IAM
reserved / commit	steady inference fleet	paid used or not
capacity block / calendar	deadline training	prepaid, **not a discount**
spot / preemptible	checkpointed batch (50–70% off)	eviction: AWS 2 min, Azure and GCP ~30 s
flex start	fine-tunes, batch	you wait

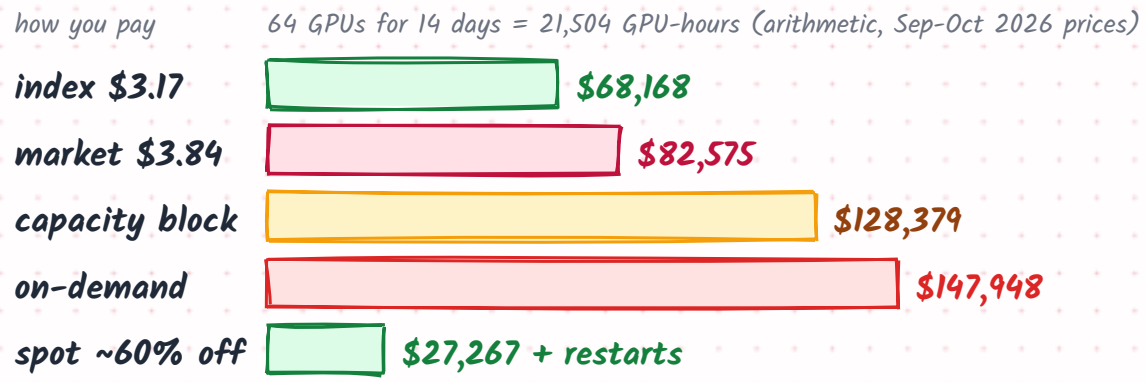
6 The commitment rule

> A commitment at discount d wins **only if** use of the committed capacity is above $1 - d$. "A discount of d buys you the right to waste d."

Discount	Break-even use	Check (arithmetic)
30%	> 70%	60% use: on-demand wins
40%	> 60%	50% loses, 70% wins
55% (3-year)	> 45%	35% loses, 55% wins

7 One training run, five ways to pay

64 H100 x 14 days x 24 h = **21,504 GPU-hours**. Prices approximate, Sep-Oct 2026.



the same job costs up to 5.4 times more, depending on how you buy

Spread: **5.4x**. Spot excludes lost steps and restarts (lesson 2). AWS Capacity Blocks rose +15%, +20%, +15% in 2026 = **1.59x** in a year; a block booked before a hike keeps its rate. Use 2-3 providers and a portable stack; shorter commit terms limit lock-in above market.

	Neocloud	Hyperscaler
price per GPU-hour	lowest	highest
new chips	first	later
data already there	egress to move it	no egress
identity, compliance, managed services	thinner	deep

8 Price every token type

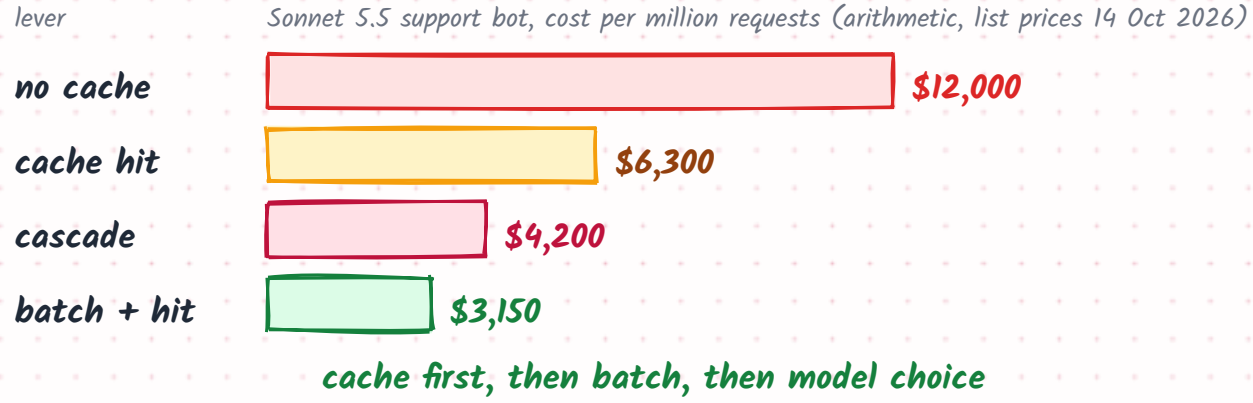
> **cost per request** = uncached input + cache writes + cache reads + **output** (incl. thinking). Batch x 0.5. Some regional options x 1.1.

$ per million tokens (list, 14 Oct 2026)	Input	Cache read	Output
Opus 5.5	$4	$0.20	$20
Sonnet 5.5	$2	$0.10	$10
Haiku 5.5	$0.10	$0.01	$0.50

Same shape everywhere: output about 5-6x input, cache reads 0.1x or less, batch half. Notebook figures; check the vendor page before budgeting.

9 A support bot, costed

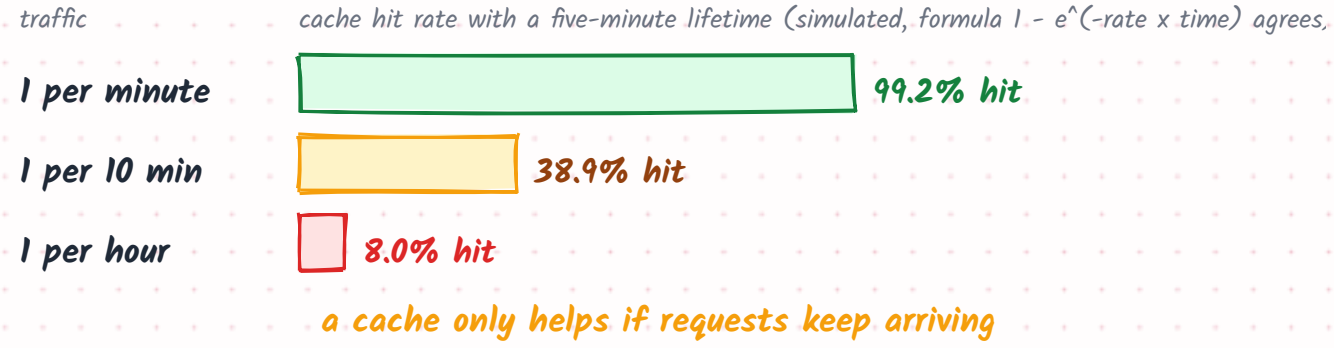
3,000 cacheable + 1,000 uncached input tokens, 400 output. Script: l11_token_cost.py.



10 Lever 1: caching

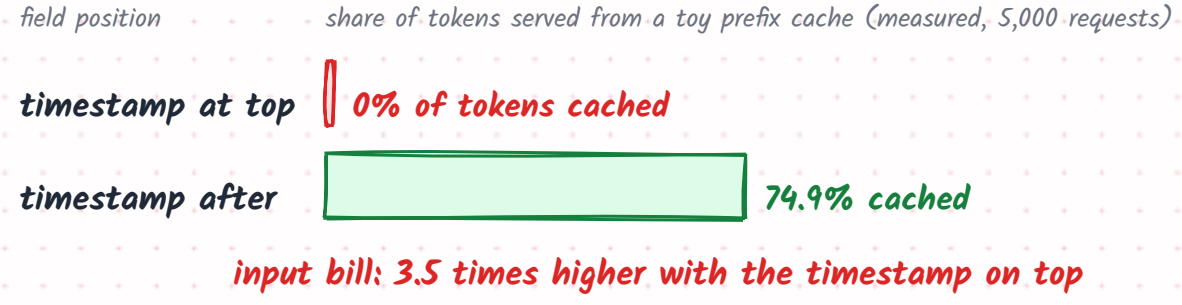
- ✓ 5-minute write 1.25x input, read 0.05-0.1x: pays back after **one read** (1-hour: after two)
- ✓ Cache hit saves **47.5%** per request (Sonnet 5.5)
- ✓ Hit rate 50%: $9,900 · 80%: $7,740 · 95%: $6,660 per 1M requests

11 Traffic decides the hit rate



Hit rate = 1 − e^(−rate x lifetime). Simulated random arrivals agreed with the formula within a point. A quiet feature can pay for writes and never get the reads.

12 The timestamp mistake



13 Measured: a toy prefix cache

A tiny prefix cache I wrote (not any vendor's), 5,000 requests of about 4,000 tokens, one changing field.

Where the changing field sits	Tokens served from cache	Input bill
top of the system prompt	0%	3.5x
after the shared part	74.9%	1x

Stable text first, changing text last. Put timestamps, user names and request ids after the cached prefix. Alert on cache-read share: a sudden drop means a silent cache break.

14 Levers 2 to 5

Lever	What	Note
2 Batch	50% off every token incl. cached, results within 24 h	offline evals, backfills, labelling
3 Audit	trim input: tool search -45%, prune stale results -39%, audit -14%	published tests
4 Effort / model	low effort first, re-run failures at high: about -40%	needs a reliable failure signal
5 Routing	Haiku first, 30% to Sonnet: $4,200 vs $12,000 (-65%)	classifier must be reliable; escalations pay twice

15 Output, tasks and tokenizers

- → 400 to 250 output tokens saves **$1,500** per 1M requests (12.5% of the uncached bill)
- → Trim with format instructions or structured output, **not** a tiny max_tokens (it truncates)
- → Compare **cost per solved task**, not $/MTok: new tokenizers make about 30% more tokens for the same text
- → Blended price = 0.8 x input + 0.2 x output. Agents are often over 90% input: caching dominates

> Lever order **C-B-A-E-M**: Cache and trim, Batch, Audit, Effort / model, Multi-model last. "Cheap Bots Always Earn Money."

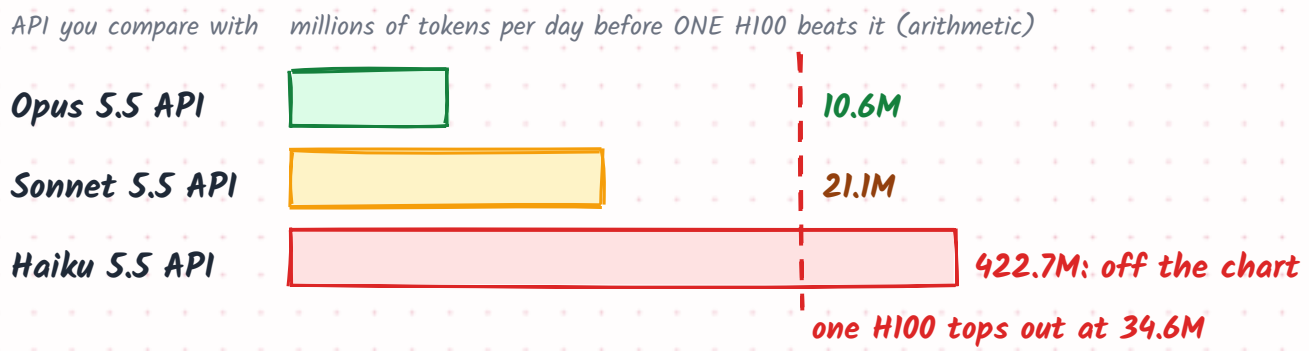
16 Self-host cost per million tokens

$/MTok = GPU $/hr ÷ (tok/s x 3,600 x utilisation) x 10^6. 70B FP8, one H100 at $3.17, about 400 tok/s (vendor figure, likely conservative). Script: l11_breakeven.py.

Utilisation	$ per MTok
100%	$2.20
60%	$3.67
40%	$5.50
15%	$14.68

17 The break-even volume

One H100 24/7 = **$76.08 a day**, capacity about **34.6M tokens/day**. Break-even = GPU $/day ÷ API blended $/MTok.



API	Blended $/MTok	Break-even	Verdict
Opus 5.5	$7.20	10.6M/day	self-host can win above this
Sonnet 5.5	$3.60	21.1M/day	narrow window up to 34.6M
Haiku 5.5	$0.18	422.7M/day	never on one GPU
Sonnet, GPU at $5.97	$3.60	39.8M/day	never on one H100

18 What the sum hides

Simulated year of bursty traffic, mean 20M tokens/day, one GPU for the base load and the API for overflow.

Option	Cost per year
all API (Sonnet 5.5)	$31,909
1 GPU + API overflow	$31,678

GPU used 62% of capacity, 12.2% of tokens overflowed. A tie **before** on-call, upgrades, spare replicas and any quality gap of an open model. Price a hosted open-weight endpoint and a batch + cache API first.

- ✓ Self-hosting tends to win: steady high volume on small or fine-tuned models
- ✓ strict data residency, or latency an API cannot meet
- ✓ GPUs already owned or committed with idle headroom
- ✓ Serve the base load yourself, overflow to the API

19 Who spent it?

```
# one gateway log line per LLM call (minimum fields)
{"trace_id": "conv-8f3a", "team": "support", "feature": "triage-bot",
 "tenant_id": "acme", "model": "sonnet-5.5", "uncached_in": 1000,
 "cache_read": 3000, "output": 400,
 "price_table": "2026-10-14", "cost_usd": 0.0063}
```

→ GPUs: cloud tags + Kubernetes labels + an allocation tool

→ Tokens have no tags: gateway stamps, per-project keys or workspaces, provider usage APIs

→ Capture usage when the request happens; join agent fan-out by trace id

→ Compute cost at log time from a **versioned** price table; reconcile monthly

20 Showback and cost per resolved ticket

Feature (invented logs)	Cost / day	Resolve rate	Per resolved ticket
triage-bot	$26.70	70%	$0.057
order-lookup	$6.41	95%	$0.009
refund-agent	$113.79	45%	$0.165

Per resolved = cost per request x turns ÷ resolve rate (6 turns assumed, 2 for lookups). Showback first; chargeback once allocation rules are stable.

21 Budgets, caps and alerts

- ✓ Monthly budget per workspace or key; alerts at 50 / 80 / 100%
- ✓ Hard spend cap at the gateway (stops a runaway agent)
- ✓ Anomaly rule: daily cost above mean + 3 sigma of the last 14 days
- ✓ Also alert on cache-read share

Test on **invented logs**: on day 45 a timestamp moved to the top of one feature's prompt. Daily cost rose **1.8x**, cache share fell **70% to 0%**, both alerts fired that day, 0 false alarms in the two healthy features. Script: l11_gateway_anomaly.py.

22 Hidden lines: checkpoints



Cross-AZ about $0.02/GB. A 70B full checkpoint is 1,120 GB: $22.40 to move, $538/day if hourly. Keep the last 3 full, weights-only daily, expire or archive the rest. Keep GPUs, data and checkpoints in one zone.

23 ShopNest Copilot: cost plan

Lever	Plan
cache	stable prompt first, changing fields last, alert on cache-read share
batch + effort	evals and labelling in batch; low effort, escalate on a failing check
attribution	gateway stamps; cost per resolved ticket; cap per key
GPUs	busy or not rented; price a hosted open model first

24 Words to keep

Word	Meaning
Utilisation	share of paid hours doing useful work
Real price	list price ÷ utilisation
Cache hit	prompt prefix already stored, billed at a fraction
Unit cost	cost of one useful outcome, like a resolved ticket

25 Interview corner

Q. How do you cut the cost of an AI system?

A. Fix idle capacity first (real price = list price ÷ utilisation), then cache, batch, prompt audit, effort and routing last. In my toy test a changing field on top of the prompt gave 0% cache hits.

Q. Self-host or API?

A. Compute the break-even volume. One H100 at $3.17/hr beats a premium model above roughly 11 to 21 million tokens a day, and never beats a Haiku-class API, so judge cost per solved task.

Q. When is a commitment worth it?

A. When use of the committed capacity stays above 1 minus the discount: at 40% off, above 60%.

Q. How do you attribute AI spend?

A. A gateway stamps team, feature, tenant and trace on every call, with a versioned price table; report cost per resolved ticket.

26 Your turn

111_gpu_price.py · **111_token_cost.py** · **111_breakeven.py** · **111_gateway_anomaly.py**: put in your own prices and utilisation.

⚡ Revise in 60 seconds

- → real price = list ÷ use: $3.17 at 5% = **$63.40**
- → commit only if use > **1 − d**
- → one run, five ways: **5.4x** spread
- → cache → batch → audit → effort → route
- → timestamp on top: **0%** cached vs 75%
- → one H100 never beats Haiku-class API
- → break-even vs Sonnet: **21.1M** tokens/day
- → cost per resolved ticket

Next lesson → CAI-12
Your learning path: from zero to cloud AI engineer
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

CLOUD NOTEBOOK